

Guide du développeur

Agile Practice Modeler (Agilia)

Mohamed AIT HASSOU & Nicolas MATTON (Client)

18 avril 2026

Table des matières

Par où commencer ?	3
Langages et technologies	4
Déploiement (première partie)	5
Architecture générale	5
Structure du backend	5
Structure du frontend	6
Base de données	6
Flux de données typique	6
Gestion des erreurs et logs	7
Sécurité	7
Extensions possibles	7
Principe du fil d'Ariane – Navigation top-down	7
Arborescence des classes et modules	8
Composants principaux de l'architecture	8
Exemple de diagrammes	9
Diagramme d'activité : création d'un bloc (ex. Activity)	9
Diagramme de séquence : association pratique méthode	9
Pourquoi React ?	9
Pourquoi Node.js/Express ?	10

Pourquoi PostgreSQL ?	10
Pourquoi le verrouillage par localStorage ?	10
Pourquoi l'export PNG avec html2canvas ?	10
Composants réutilisables	11
Exemples de code – complexité croissante	11
Documentation technique de l'API	12
Déploiement (sans Docker)	12

Par où commencer ?

Avant toute contribution ou reprise du projet, il est essentiel de se familiariser avec l'organisation du code, la documentation disponible ainsi que le contexte dans lequel le projet a été développé.

Où sont les sources ?

Les sources complètes sont hébergées sur un dépôt GitHub . La structure du dépôt est la suivante :

- **backend/** : contient le serveur Node.js/Express (**server.js**), le fichier **package.json** des dépendances, et un fichier d'exemple **.env** à renommer.
- **src/** : contient le code source React.
- **bin/** : contient le dossier **build/** pour la compilation de production.
- **data/** : contient le script SQL **schema.sql** qui permet de recréer la structure de la base de données PostgreSQL.
- **LICENSE** : fichier de la licence MIT.
- **README.md** : présentation rapide du projet et instructions d'installation.

Où est la documentation ?

La documentation se trouve sous plusieurs formes :

- **Code commenté** : chaque fichier JavaScript/JSX contient des en-têtes JS-Doc décrivant le rôle du module, les fonctions principales et leurs paramètres.
- **README.md** : guide d'installation rapide.
- **Ce guide développeur** : architecture, tests, déploiement.
- **Guide utilisateur** (fichier séparé) : destiné aux utilisateurs finaux.
- **Schéma de base de données** : le fichier **data/schema.sql** est auto-documenté (commentaires sur les tables et colonnes).

Qui ont été les développeurs ?

Mohamed AIT HASSOU dans le cadre d'un projet académique, encadré et assisté par Monsieur Nicolas MATTON.

Où est le cahier des charges ?

Le cahier des charges initial correspond à l'énoncé du projet AMQuICk (chapitres 8.4 et suivants). Les objectifs fonctionnels sont :

- Modéliser des pratiques agiles sous forme de cartes graphiques.
- Permettre le glisser-déposer et la sauvegarde des positions.
- Intégrer un référentiel de pratiques via web services.
- Gérer les profils Big Five et les enquêtes d'affinité.
- Offrir une interface d'administration pour les experts (associations, création de méthodes, etc.).

Environnement de développement initial

- Système d'exploitation : Windows 11

- Éditeur : Visual Studio Code
- Serveur local : Node.js 18 LTS, npm 9, PostgreSQL 14
- Navigateur : Microsoft Edge pour les tests d'interface
- Outils : Git, Postman

Langages et technologies

Composant	Langage / Framework
Backend API	Node.js / Express
Frontend	React
Base de données	PostgreSQL
Authentification	bcrypt (hash)
Export d'image	html2canvas
Icônes	lucide-react
Tests backend	Jest + Supertest

TABLE 1 – Technologies utilisées

Compilation du frontend (React)

Les sources du frontend se trouvent dans le dossier `src/`. Pour produire une version optimisée pour la production :

```
1 cd projet      # ou le dossier contenant le frontend
2 npm install    # installe les dependances
3 npm run build
```

Listing 1 – Compilation du frontend

Les fichiers compilés sont alors générés dans le dossier `build/` (ou `bin/build/`). Ce dossier contient l'application prête à être servie par un serveur HTTP statique.

Backend (Node.js) – aucune compilation

Le backend est écrit en JavaScript standard. Il suffit d'installer les dépendances et de lancer le serveur :

```
1 cd backend
2 npm install
3 node server.js
```

Listing 2 – Lancement du backend

Le serveur démarre sur le port configuré (5000 par défaut).

Compilation complète pour un déploiement local

- Base de données : exécutez `schema.sql` dans PostgreSQL.
- Backend : `node server.js` (ou `nodemon server.js`).
- Frontend : `npm start` (développement) ou `npm run build` puis `npx serve -s build`.

À propos des tests

Le backend contient des tests d'intégration (dossier `backend/routes.test.js`).
Exécution :

```
1 cd backend
2 npm test
```

Les tests utilisent Jest et Supertest ; ils nécessitent une base de données de test (à configurer).

Déploiement

Préparer la base de données

1. Lancer PostgreSQL.
2. Créer une base de données (ex. `agilia`).
3. Exécuter le script `projet/data/schema.sql` dans cette base.

Démarrer le backend

```
1 cd projet/backend/
2 npm install
3 node server.js
```

Le backend tourne sur `http://localhost:5000`.

Servir le frontend Le frontend compilé se trouve dans `bin/build/`. Avec Node.js, installer `serve` :

```
1 npm install -g serve
2 serve -s bin/build -l 3000
```

Architecture générale

L'architecture du logiciel est de type client-serveur monolithique. Le frontend React communique exclusivement avec l'API backend via des requêtes HTTP échangeant des données au format JSON. Le backend, construit avec Express, expose une API REST et dialogue avec la base de données PostgreSQL en SQL. Le serveur ne délivre pas de pages HTML : toutes ses routes sont des endpoints REST. Cette séparation stricte permet une évolution indépendante des deux couches et facilite l'intégration avec d'autres clients.

Structure du backend

`server.js` est le point d'entrée unique définissant plus de 80 routes. Il utilise :

- Un pool de connexions PostgreSQL (`pg`)
- `dotenv` pour les secrets
- Routes organisées par famille : pratiques, versions, activités, rôles, recommandations, etc.

Extension possible : pour ajouter une nouvelle entité (ex. « Outil ») :

1. Créer la table SQL dans `schema.sql`.
2. Ajouter les routes POST/GET/DELETE dans `server.js`.
3. Ajouter un type de bloc dans le frontend.

Structure du frontend

Les composants principaux sont :

- `EspaceTravail.js` : canevas graphique (glisser-déposer, liens SVG, menus contextuels) – cœur de l'application.
- `Practices.js` : affichage des pratiques et versions d'une équipe.
- `Teamdetails.js` : page d'équipe.
- `Dashboard.js` : accueil après connexion.
- `MonProfil.js` : questionnaire d'affinité.
- `Login.js`, `Inscription.js` : authentification.
- `Addpractice.js`, `Practiceversionform.js` : formulaires.

Les styles sont dans `src/styles/` (CSS simples).

Exemple : Ajouter un nouveau type de bloc

1. Modifier `EspaceTravail.js` : ajouter une entrée dans `pieces_pour_essay` (objet avec `id`, `type`, `couleur`, `nom`).
2. Dans la fonction `renderPiece`, ajouter un `case` pour le nouveau type.
3. Dans la fonction `validerform`, ajouter un `switch case` pour l'enregistrement en base (créer une route backend).
4. Créer une nouvelle table SQL et les routes CRUD.

Base de données

Le schéma est entièrement décrit dans `data/schema.sql`. Tables principales :

- `Practice`, `PracticeVersion`
- `Activity`, `Role`, `Workproduct`, `Goal`, `Metric`, `Guideline`, `Pitfall`, `Benefit`, `Context`, `CompletionCriteria`, `Recommendation`
- Tables de liaison : `PracticeVersionActivity`, `PracticeVersionRole`, etc. (stockent positions `x`, `y` et séquences)
- `Person`, `Team`, `TeamMember`, `Universe`
- `AffinitySurvey`, `AffinitySurveyVersion`, `AffinityPractice`, `AffinitySurveyResults`
- `BfProfile`, `BfProfileStatus`
- `PracticeAssociation`, `PracticeMethod`

Point d'attention : Toutes les modifications de schéma doivent être répercutées dans `schema.sql` et versionnées.

Flux de données typique

- **Création d'un bloc** : glisser → formulaire → POST /activity → réponse avec ID → mise à jour du state React → affichage.

- **Déplacement** : événements souris → mise à jour locale → PUT /activity-position avec nouvelles coordonnées.
- **Liens SVG** : calculés dynamiquement par `EspaceTravail` via `getCenter`, dessinés dans une couche SVG.

Gestion des erreurs et logs

Dans l'état actuel du développement, la gestion des erreurs côté backend est assurée par l'utilisation de la fonction `console.error` qui permet d'afficher dans la console les problèmes rencontrés lors de l'exécution des requêtes. Côté frontend, les erreurs sont signalées à l'utilisateur au moyen de boîtes de dialogue `alert()`.

Sécurité

La protection des comptes utilisateurs repose sur le hachage des mots de passe avec la bibliothèque `bcrypt` et un facteur de coût fixé à 10, ce qui garantit une bonne résistance aux attaques. Les échanges entre le frontend et le backend s'effectuent actuellement en HTTP. Pour un déploiement en production, il est prévu de passer en HTTPS, ce qui chiffrera les communications et renforcera la confidentialité des données transmises.

Extensions possibles

Plusieurs évolutions peuvent être envisagées pour étendre les fonctionnalités du logiciel. L'ajout de WebSockets permettrait une collaboration en temps réel entre plusieurs utilisateurs sur un même espace de travail. Des fonctions d'import et d'export de pratiques aux formats JSON ou XML faciliteraient l'échange de modèles avec d'autres outils. Enfin, la prise en charge d'un thème personnalisable via les variables CSS (`custom properties`) offrirait à chaque utilisateur la possibilité d'adapter l'apparence de l'interface selon ses préférences.

Principe du fil d'Ariane – Navigation top-down

1. **Besoins métier** (cahier des charges AMQuICk)
2. **Cas d'utilisation** : créer une pratique, éditer dans l'espace de travail, associer pratiques/méthodes, enquête d'affinité.
3. **Interfaces utilisateur (React)** : Dashboard, TeamDetails, EspaceTravail, Practices, MonProfil...
4. **API REST (Express)** : routes correspondant à chaque action.
5. **Persistance (PostgreSQL)** : tables SQL et requêtes paramétrées.

Arborescence des classes et modules

Backend

```
backend/  
  server.js  
  package.json  
  .env  
  routes.test.js
```

Motivation : Toutes les routes dans un seul fichier pour simplifier le développement. Une refactorisation en couches (controllers, services, repositories) serait nécessaire si le projet grossit.

Frontend

```
src/  
  pages/                                # Composants par route  
    Dashboard.js  
    Teamdetails.js  
    EspaceTravail.js  
    Practices.js  
    MonProfil.js  
    Login.js  
    Inscription.js  
    Addpractice.js  
    Practiceversionform.js  
    Creerteam.js  
    Affichageteams.js  
    Practicevspractice.js  
    Practicemethode.js  
  styles/  
  App.js  
  index.js  
  assets/
```

Motivation : Cette organisation par pages permet une structuration claire et intuitive de l'application, facilitant la compréhension et la navigation pour les développeurs, en particulier les débutants en React. La séparation des styles améliore la lisibilité et la maintenabilité du code

Composants principaux de l'architecture

Le client, développé avec React, a pour rôle d'assurer l'interface utilisateur, la gestion de l'état local via les hooks, les appels HTTP vers l'API et le rendu des éléments graphiques, y compris les SVG. Il s'appuie sur plusieurs bibliothèques : react et react-dom pour le cœur de l'interface, react-router-dom pour la navigation entre les pages, html2canvas pour l'export d'images du canevas, et lucide-react pour les

icônes. Ce choix est motivé par la flexibilité de React, la richesse de son écosystème et la maturité de `html2canvas` qui capture fidèlement le DOM.

Le serveur, construit avec Express, reçoit les requêtes HTTP, dialogue avec PostgreSQL et renvoie des réponses au format JSON. Les bibliothèques utilisées sont `express` pour le framework web, `pg` comme client PostgreSQL avec pool de connexions, `cors` pour autoriser les requêtes cross-origin en développement, `dotenv` pour la gestion des variables d'environnement, et `bcrypt` pour le hachage des mots de passe. Express a été préféré pour sa légèreté et sa large adoption ; `pg` est le client PostgreSQL de référence.

La décomposition logique du serveur, bien que non matérialisée physiquement dans des dossiers séparés, distingue trois couches : l'interface clients (les routes Express), la couche business (les handlers qui contiennent les règles métier), et la couche de persistance (les requêtes SQL exécutées via `pg`). Cette organisation est justifiée par la taille modérée du projet (environ 80 routes) : une séparation physique en dossiers `controllers`, `services` et `repositories` aurait ajouté une complexité inutile.

Exemple de diagrammes :

Diagramme d'activité : création d'un bloc (ex. Activity)

L'utilisateur glisse le bloc « Activity » depuis la barre d'outils vers le canevas. Le composant `EspaceTravail` détecte le relâchement de la souris, ouvre une modale de formulaire. Après validation, le frontend envoie une requête `POST /activity`. Le backend insère une ligne dans `Activity`, puis dans `PracticeVersionActivity` avec les coordonnées et le numéro de séquence. Il retourne l'ID. Le frontend ajoute le bloc dans son état local, provoquant le rafraîchissement. Les liens SVG sont recalculés. Le bloc devient déplaçable.

Diagramme de séquence : association pratique méthode

Un Expert, depuis la page d'une équipe, clique sur « Associer pratique méthode ». Une modale affiche trois colonnes : pratiques, méthodes, versions de la méthode sélectionnée. L'Expert sélectionne une version de pratique, puis une méthode, puis une ou plusieurs versions de cette méthode. En cliquant « Associer », le frontend envoie une requête `POST /practiceMethod` avec l'ID de la version de pratique et un tableau d'IDs de versions de méthode. Le backend exécute des `INSERT` dans `practiceMethod`. Réponse HTTP 200. Fermeture de la modale et message de confirmation.

Pourquoi React ?

React a été choisi pour le frontend en raison de sa courbe d'apprentissage douce, qui permet à un développeur de devenir rapidement opérationnel. La notion de com-

posants réutilisables facilite la construction d'interfaces complexes comme l'espace de travail graphique, où chaque bloc (activité, rôle, etc.) peut être encapsulé et réutilisé. La gestion d'état via les hooks (useState, useEffect) rend le comportement de l'application prévisible, notamment pour le suivi des positions des blocs et des liens SVG. Enfin, l'écosystème React offre des bibliothèques matures comme html2canvas pour l'export d'images, qui s'intègre parfaitement avec le rendu DOM.

Pourquoi Node.js/Express ?

Le choix de Node.js avec Express repose sur l'uniformité du langage : utiliser JavaScript aussi bien côté serveur que côté client réduit la charge cognitive et permet le partage éventuel de code. Express est un framework minimaliste, très léger, qui convient parfaitement à la création d'une API REST sans surcharge inutile. Sa grande popularité garantit une documentation abondante et une facilité de recrutement. De plus, l'approche événementielle et non bloquante de Node.js est bien adaptée aux opérations d'entrée/sortie (accès base de données, appels réseau) que réalise le backend.

Pourquoi PostgreSQL ?

PostgreSQL a été retenu pour la persistance des données en raison de la nature fortement relationnelle du domaine : les pratiques, versions, activités, rôles et leurs nombreuses liaisons exigent un modèle relationnel avec des clés étrangères et des contraintes d'intégrité référentielle. PostgreSQL garantit la cohérence des données, par exemple lors de la suppression en cascade d'une version de pratique. Par ailleurs, le stockage des coordonnées (x, y) des blocs et la gestion des séquences sont parfaitement pris en charge par ses capacités de requêtes spatiales et son support natif des types numériques. Enfin, sa conformité aux standards SQL et ses performances en font un choix robuste pour une application de modélisation.

Pourquoi le verrouillage par localStorage ?

Le mécanisme de verrouillage des cartes graphiques repose sur localStorage pour des raisons de simplicité. Le besoin fonctionnel est uniquement d'éviter que deux utilisateurs modifient simultanément la même pratique, ce qui entraînerait des conflits de sauvegarde. Il n'est pas nécessaire d'implémenter une collaboration en temps réel avec des WebSockets, plus complexe à mettre en œuvre. localStorage permet de stocker un jeton de verrouillage avec un horodatage directement dans le navigateur, sans appel serveur supplémentaire, ce qui suffit largement pour un usage où les modifications sont peu fréquentes et où l'utilisateur est informé en cas de conflit.

Pourquoi l'export PNG avec html2canvas ?

La bibliothèque html2canvas a été choisie pour l'export d'images du canevas graphique en raison de sa facilité d'intégration : une seule fonction permet de capturer l'intégralité d'un élément DOM et de le transformer en image PNG. Elle offre une

qualité de rendu fidèle au navigateur, respectant les CSS, les polices et surtout les éléments SVG qui constituent les liens entre blocs. Ainsi, l'utilisateur obtient une image exacte de sa carte de pratique, telle qu'il la voit à l'écran. Aucune autre bibliothèque ne fournit un équivalent aussi simple et aussi bien adapté à une application React.

Composants réutilisables

Agilia n'est pas livré comme bibliothèque npm, mais certains modules peuvent être réutilisés :

- **Backend** : importer `server.js` comme module Express et monter sur un sous-chemin.

```
1 const agiliaApp = require('./backend/server');
2 app.use('/agilia', agiliaApp);
```

- **Frontend** : Exemple : intégrer EspaceTravail dans une autre application React.

```
1 import EspaceTravail from './pages/EspaceTravail';
2 <Route path="/mon-app/agilia/:practiceVersionId" element={<
  EspaceTravail />} />
```

- **API client** : appeler directement l'API REST depuis n'importe quel langage.

Exemples de code – complexité croissante

Niveau 1 – Récupérer les pratiques d'une équipe (curl)

```
1 curl -X GET http://localhost:5000/teams/1/pratiques
```

Niveau 2 – Créer une pratique et une version (JavaScript fetch)

```
1 const practice = await fetch('http://localhost:5000/practice', {
2   method: 'POST',
3   headers: { 'Content-Type': 'application/json' },
4   body: JSON.stringify({
5     name: 'Test pratique',
6     description: 'Description',
7     objective: 'Objectif',
8     typeId: 1
9   })
10 }).then(r => r.json());
11
12 const version = await fetch('http://localhost:5000/practiceVersion', {
13   method: 'POST',
14   headers: { 'Content-Type': 'application/json' },
15   body: JSON.stringify({
16     practiceId: practice.id,
17     universeId: 1,
18     versionName: 'v1.0',
19     changeDescription: 'Version initiale',
20     lastUpdateById: 1
21   })
22 }).then(r => r.json());
```

```

23
24 console.log('Espace de travail :', '/espace-travail/${version.id}')
    ;

```

Niveau 3 – Ajouter un bloc Activity avec position

```

1 const activity = await fetch('http://localhost:5000/activity', {
2   method: 'POST',
3   headers: { 'Content-Type': 'application/json' },
4   body: JSON.stringify({
5     name: 'Demo',
6     description: 'Bloc ajoute par script',
7     lastUpdateById: 1,
8     practiceVersionId: 42,
9     sequence: 1,
10    x: 100,
11    y: 200
12  })
13 });

```

Niveau 4 – Calcul du profil Big Five

```

1 await fetch('http://localhost:5000/bfProfile/calculate/7', { method
  : 'POST' });
2 const profile = await fetch('http://localhost:5000/bfProfile?
  personId=7').then(r => r.json());

```

Documentation technique de l'API

Ressource	Endpoint	Méthode	Description
Pratiques	/pratiques	GET	Liste toutes les pratiques
Pratique	/practice	POST	Crée une pratique
Version	/practiceVersion	POST	Crée une version de pratique
Activités	/practiceVersion/ :id/activities	GET	Récupère les activités d'une version
Activité	/activity	POST	Ajoute une activité
Position activité	/practiceVersion/ :id/activity-position	POST	Met à jour les coordonnées (x, y)
Connexion	/login	POST	Authentification
Inscription	/register	POST	Crée un compte
Profil Big Five	/bfProfile?personId=X	GET	Récupère le profil
Calcul Big Five	/bfProfile/calculate/ :personId	POST	Calcule le profil
... (88 routes du server.js)			

Déploiement

Le projet n'a pas été déployé à l'aide de conteneurs tels que Docker. Il est conçu pour une exécution en local dans un environnement de développement classique.

Configuration minimale pour usage local : 2 cœurs, 4 Go RAM, 1 Go disque.